setup code for the thread executing `push()` depends on the interface to the queue and the type of object being stored. If the object being stored is expensive to construct or must be heap allocated, you want to do this as part of the thread-specific setup, so that it doesn't affect the test. On the other hand, if the queue is just storing plain `int`s, there's nothing to be gained by constructing an `int` in the setup code. The actual code being tested is relatively straightforward—a call to `push()` from one thread and a call to `pop()` from another—but what about the "after completion" code?

In this case, it depends on what you want `pop()` to do. If it's supposed to block until there is data, then clearly you want to see that the returned data is what was supplied to the `push()` call and that the queue is empty afterward. If `pop()` is *not* blocking and may complete even when the queue is empty, you need to test for two possibilities: either the `pop()` returned the data item supplied to the `push()` and the queue is empty or the `pop()` signaled that there was no data and the queue has one element. One or the other must be true; what you want to avoid is the scenario that `pop()` signaled "no data" but the queue is empty, or that `pop()` returned the value and the queue is *still* not empty. In order to simplify the test, assume you have a blocking `pop()`. The final code is therefore an assertion that the popped value is the pushed value and that the queue is empty.

Now, having identified the various chunks of code, you need to do the best you can to ensure that everything runs as planned. One way to do this is to use a set of `std::promise`s to indicate when everything is ready. Each thread sets a promise to indicate that it's ready and then waits on a (copy of a) `std::shared_future` obtained from a third `std::promise`; the main thread waits for all the promises from all the threads to be set and then triggers the threads to `go`. This ensures that each thread has started and is just before the chunk of code that should be run concurrently; any thread-specific setup should be done before setting that thread's promise. Finally, the main thread waits for the threads to complete and checks the final state. You also need to be aware of exceptions and make sure you don't have any threads left waiting for the `go` signal when that's not going to happen. The following listing shows one way of structuring this test.